

体系结构 第三章

by xjc

第三章 流水线技术

3.1 流水线的基本概念

3.1.1 什么是流水线

(1) 流水线技术的概念

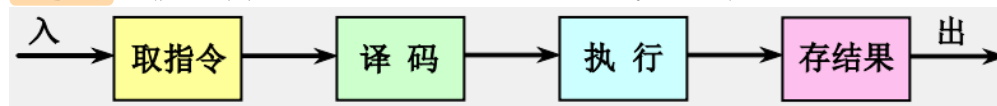
1. 把一个重复的过程分解为若干个子过程，每个子过程由专门的功能部件来实现
2. 多个处理过程在时间上错开，依次通过各功能段，这样，每个子过程就可以与其他的子过程并行进行
3. 流水线的级/段：每个子过程及其功能部件
4. 流水线的深度：流水线的段数

(2) 指令流水线

实现方案：指令解释过程分为分析和执行两个子过程，并各自使用独立的分析部件和执行部件实现

效果：速度提高一倍

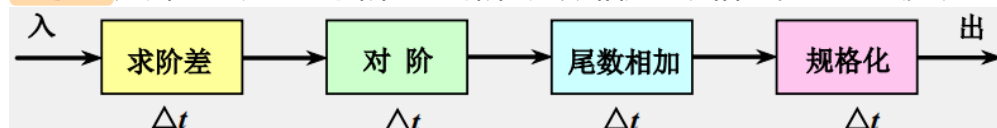
例子：四级指令流水线：IF→ID→EX→Mem，理想是4倍？



(3) 浮点加法流水线（运算操作流水线/部件级流水线）

实现方案：运算操作流水线，也成为部件级流水线

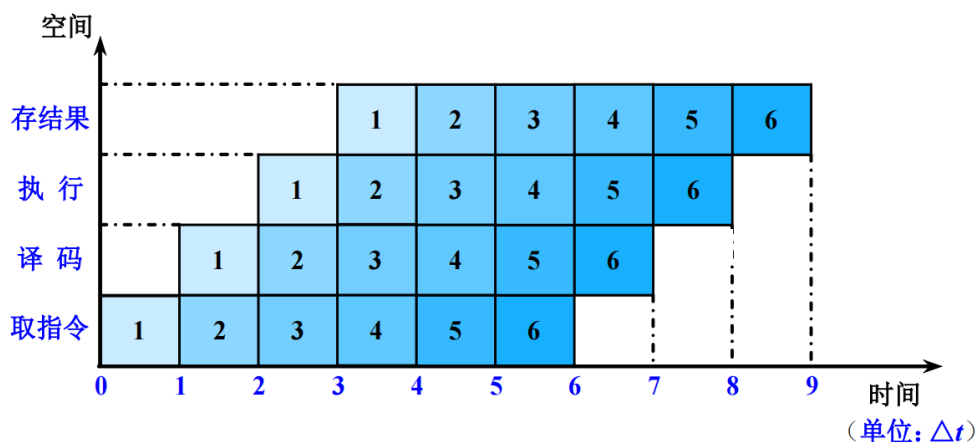
例子：分解浮点加法=求阶差+对阶+尾数相加+规格化，理想速度提高3倍



(4) 时空图

描述方法：时间和空间维度描述流水线工作

坐标：横轴——时间，纵轴——流水线各段



(5) 流水技术的特点

- 实现过程:** 流水线把一个处理过程分解为若干的子过程, 每个子过程由一个专门的功能部件来实现
 - 各段时间:** 流水线中各段的时间应尽可能相等, 否则将引起流水线阻塞、断流 (时间长的段将成为瓶颈)
 - 流水寄存器:** 流水线的每一个功能部件后都要有一个缓冲寄存器 (锁存器), 称为流水寄存器 (作用: 在相邻的两段之间传送数据, 保证提供后面要用到的数据, 并把各段的处理工作隔离)
 - 应用场景:** 适用于大量重复的时序过程, 只有在输入端不断地提供任务, 才能充分发挥流水线的效率
 - 通过和排空时间:** 流水线需要有通过时间 (第一个任务进入流水线到流出的时间) 和排空时间 (最后一个任务进入并流出的时间)
- 补充:**
- 流水线无法帮助解决单个任务的延迟, 有利于减少整个工作的全部时间
 - 多个任务同时操作需要不同的资源
 - 流水线的速率受最慢的流水线段的限制
 - 流水线各段长度不均会降低加速比
 - 充满/排空流水线的时间影响加速比
 - 流水线会由于依赖而产生阻塞

3.1.2 流水线的分类

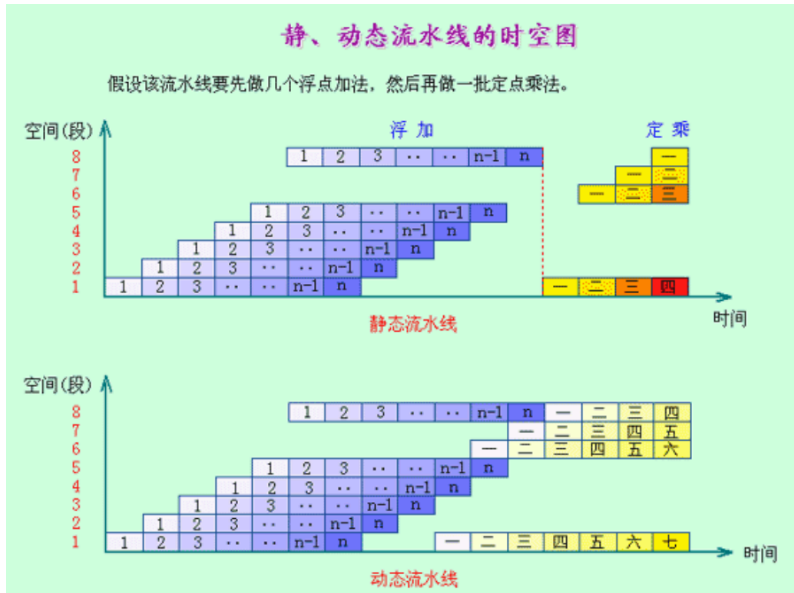
(1) 单功能/多功能流水线 (根据流水线功能分类)

- 单功能流水线:** 只有一种固定功能
- 多功能流水线:** 各段可不同连接, 实现不同功能

(2) 静态/动态流水线 (多功能流水线基础上, 各段连接方式的分类)

- 静态流水线:** 同一时间内, 只有一种功能的一种连接方式

2. **动态流水线**: 同一时间内, 可实现不同连接方式, 执行不同功能 (优缺点: 灵活, 但控制复杂)



(3) 部件级、处理机级、处理机间流水线 (按流水的级别分类)

1. **部件级流水线** (运算操作流水线): 算术逻辑运算部件分段, 实现运算操作流水
2. **处理机级流水线** (指令流水线): 指令按照多级流水方式处理
3. **处理机间流水线** (宏流水线): 两个或以上处理机串行, 对同一数据流进行处理

(4) 线性流水线、非线性流水线 (按是否有反馈回路分类)

1. **线性流水线**: 串行连接无反馈回路, 各段最多只通过一次
2. **非线性流水线**: 串行连接有反馈回路, 各段可以通过多次

(5) 顺序流水线、乱序流水线 (按任务流入和流出顺序是否相同)

1. **顺序流水线**: 输出端任务流出的顺序与输入端任务流入的顺序 **相同**, 全程一个顺序流动
2. **乱序流水线**: 输出端任务流出的顺序与输入端任务流入的顺序可以 **不同**, 允许后进入流水线的任务先完成

(6) 标量处理机与向量流水处理机

把指令执行部件中采用了 **流水线的处理机** 成为 **流水线处理机**

1. **标量处理机**: 处理机不具有向量数据表示和向量指令, 仅 **对标量数据进行流水处理** (也有流水处理)
2. **向量流水处理机**: 具有 **向量数据表示和向量指令** 的处理机

3.2 流水线的性能指标

3.2.1 吞吐率

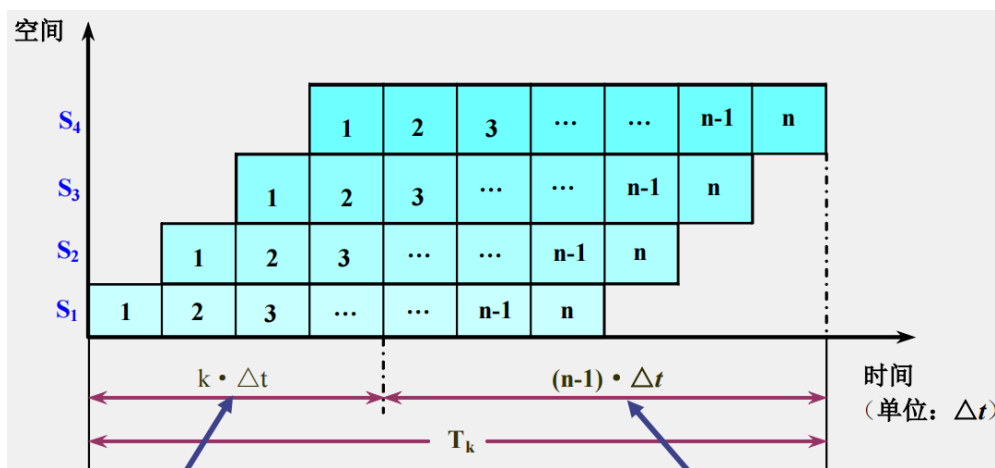
(1) 定义

吞吐率：单位时间内流水线所完成的任务数量或输出结果的数量

$$TP = \frac{n}{T_k}$$

其中 n 表示任务数， T_k 表示完成 n 个任务花费的时间

(2) 各段时间均相等的流水线



$$T_k = k\Delta t + (n-1)\Delta t$$

所以**实际吞吐率**为：

$$TP = \frac{n}{(k+n-1)\Delta t}$$

最大吞吐量 (n 无穷大时才行，一般来说， $TP < TP_{max}$)

$$TP_{max} = \lim_{n \rightarrow \infty} \frac{n}{k+n-1} = \frac{1}{\Delta t}$$

实际吞吐量与最大吞吐量的关系

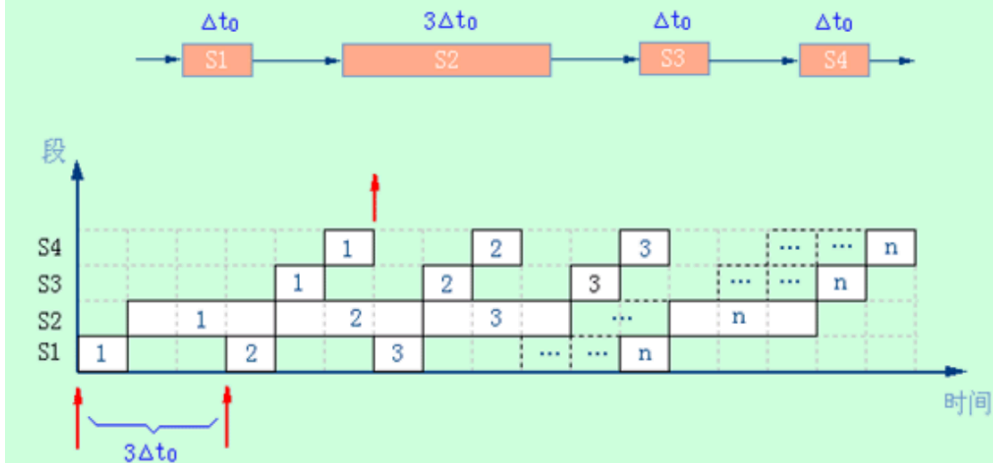
$$TP = \frac{n}{k+n-1} TP_{max}$$

(3) 各段时间不完全相等的流水线

特点：含瓶颈段（时间最长的段）

流水线的时-空图

(各段时间不等)



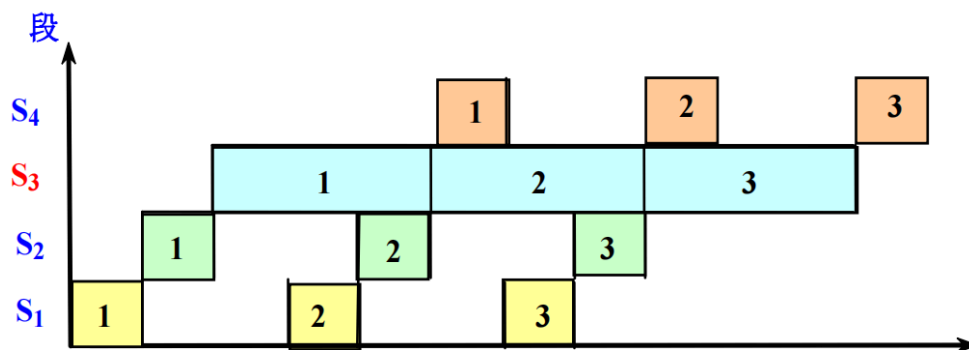
根据图可以看出，时间长度与两个因素有关：**整段流水线长度+瓶颈段长度**
假设有k的段，故**吞吐量**表达式如下，其实和相等流水线类似

$$TP = \frac{n}{\sum_{i=1}^k \Delta t_i + (n-1) \max(\Delta t_1, \Delta t_2, \dots, \Delta t_k)}$$

最大吞吐量时，n趋向于无穷大， $\sum_{i=1}^k \Delta t_i$ 可忽略，(n-1)与n相除，得到下式

$$TP_{max} = \frac{1}{\max(\Delta t_1, \Delta t_2, \dots, \Delta t_k)}$$

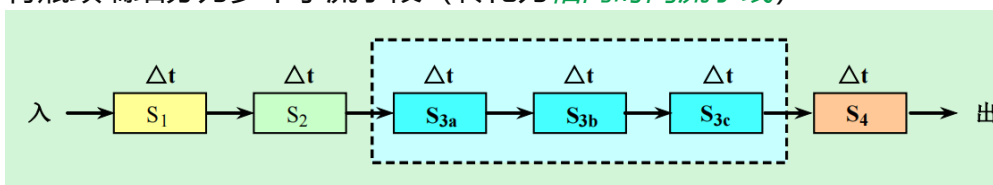
绘制时空图的关键：**瓶颈端的长度**，遵照瓶颈端绘制顺序



(4) 解决瓶颈问题的方法 (2种方法)

方法一：细分瓶颈端

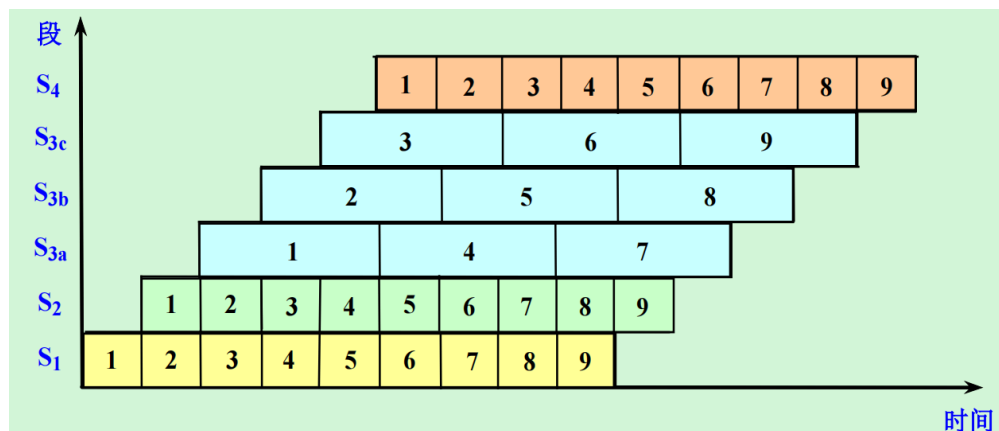
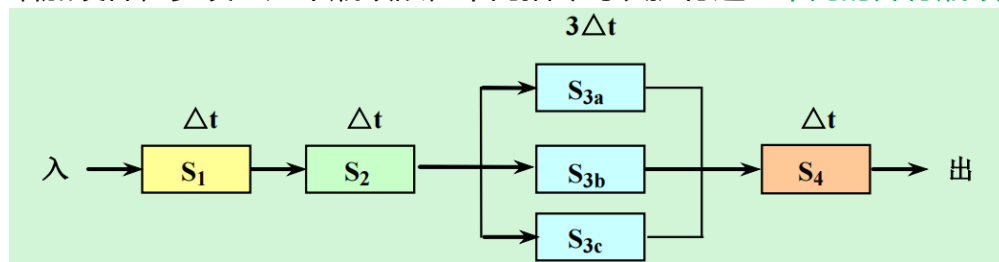
将瓶颈端细分为多个子流水段 (转化为**相同时间流水线**)



转化后的最大吞吐率 $TP_{max} = \frac{1}{\Delta t}$ ，和相同时间流水线一样

方法二：重复设计瓶颈段

增加硬件，多设置几个瓶颈段，不同指令每次执行进入不同的并行瓶颈端？



3.2.2 加速比

加速比=不使用流水线时间 T_s /使用流水线时间 T_k ，即

$$S = \frac{T_s}{T_k}$$

(1) 各段时间相等的流水线 (Δt)

不使用流水线时间 $T_s = nk\Delta t$

使用流水线时间 $T_k = k\Delta t + (n - 1)\Delta t$

故加速比为

$$S = \frac{nk\Delta t}{k\Delta t + (n - 1)\Delta t}$$

最大加速比 (n趋向无穷大) 为

$$S_{max} = k$$

(2) 各段时间不等的流水线

不使用流水线时间 $T_s = n \sum_{i=1}^k \Delta t_i$

使用流水线时间 $T_k = \sum_{i=1}^k \Delta t_i + (n - 1) \max(\Delta t_1, \Delta t_2, \dots, \Delta t_k)$

故加速比为

$$S = \frac{n \sum_{i=1}^k \Delta t_i}{\sum_{i=1}^k \Delta t_i + (n-1) \max(\Delta t_1, \Delta t_2, \dots, \Delta t_k)}$$

3.2.3 效率

定义：流水线各设备实际使用时间/总时间

Q/A：各设备的时间为什么不是总时间？各个部件流水工作，不是一直在运行的（专业的说，有通过时间和排空时间，每个部件不是一直在工作）

(1) 各段时间相等的流水线

各段效率均相等

$$e_1 = e_2 = \dots = e_k = \frac{n \Delta t}{T_k} = \frac{n}{k + n - 1}$$

总效率

$$E = \frac{e_1 + e_2 + \dots + e_k}{k} = \frac{k e_1}{k} = \frac{n}{k + n - 1}$$

最高效率（n趋向无穷大）

$$E = 1$$

(2) 效率的其他关联性与计算方法

注意各段时间相等时，吞吐率与效率成正比

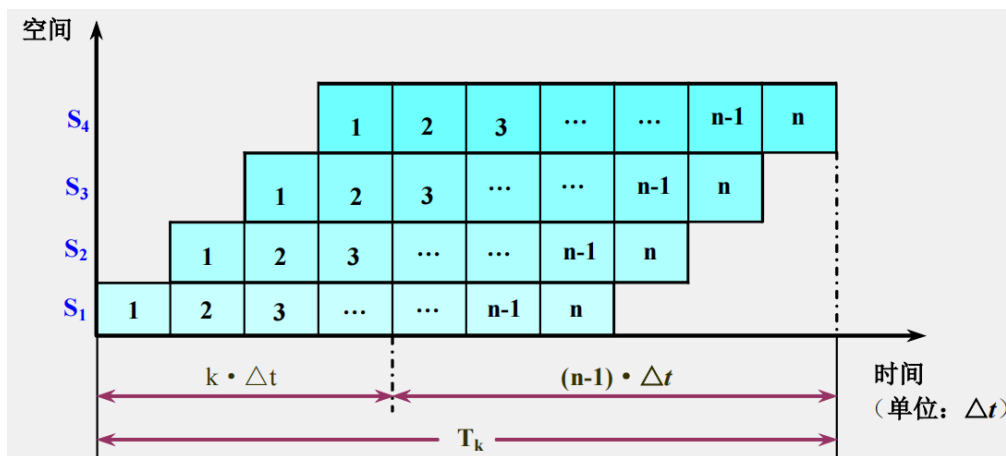
$$E = TP \Delta t$$

流水线的效率是实际加速比与最大加速比的比值

$$E = \frac{S}{k}$$

时空图角度理解效率：效率就是n个任务占用的时空图面积/k个段总的时空面积之比

$$E = \frac{n \text{个任务在时空图上的面积}}{k \text{个段的总面积}}$$



(3) 各段时间不等的流水线

$$S = \frac{n \sum_{i=1}^k \Delta t_i}{k[\sum_{i=1}^k \Delta t_i + (n-1) \max(\Delta t_1, \Delta t_2, \dots, \Delta t_k)]}$$

3.2.4 流水线的性能分析举例

着重注意第四次课ppt p36~p46，三道例题。

多功能静态流水线效率不高的可能原因：

1. 总有一些段空着
2. 功能切换时需要等前一种运算全完成
3. 运算之间的关联性导致需要等待
4. 流水线工作有建立与排空部分

3.2.5 流水线设计中的若干问题

(1) 瓶颈问题

瓶颈的产生：流水线各段不均匀时，机器的时钟周期取决于瓶颈段的延迟时间

解决方法：设计流水线时，使各段时间相当

(2) 流水线的额外开销

1. 流水寄存器延迟

流水寄存器需要建立时间和传输延迟

- 建立时间：写操作时钟信号到达前，流水寄存器输入必须保持稳定的时间
- 传输延迟：时钟信号达到后到寄存器输出可用的时间

2. 时钟偏移延迟

流水线中，时钟达到各流水寄存器的最大差值时间

(3) 流水线方式实现需要解决的问题

1. 同功能段互斥：同一时钟周期同一个功能段，不能做两份不同的工作

2. **IF和MEM冲突**: 避免IF访存（取指令）与MEM访存（读/写数据）发生冲突，可采取分离存储器或cache
3. **读写避免访存冲突**: ID段（读）和WB段（写）都要访问同一寄存器文件，解决方法是将**写操作在时钟周期前半拍完成，读操作在后半拍完成**
4. **PC的问题**:
 - PC自增必须在IF段完成
 - 分支指令可能改变PC值，导致MEM冲突
5. **其他问题**
 - 流水线**不能减少（甚至增加）单条指令的执行时间，但能够提高吞吐率**;
 - 增加流水线深度（段数）可以提高流水线的性能
 - 流水线深度受限于流水线的**额外开销**
 - 当时钟周期小至于额外开销相同时，流水已没有意义

3.3 流水线的相关与冲突

3.3.1 一个经典的5段流水线（了解）

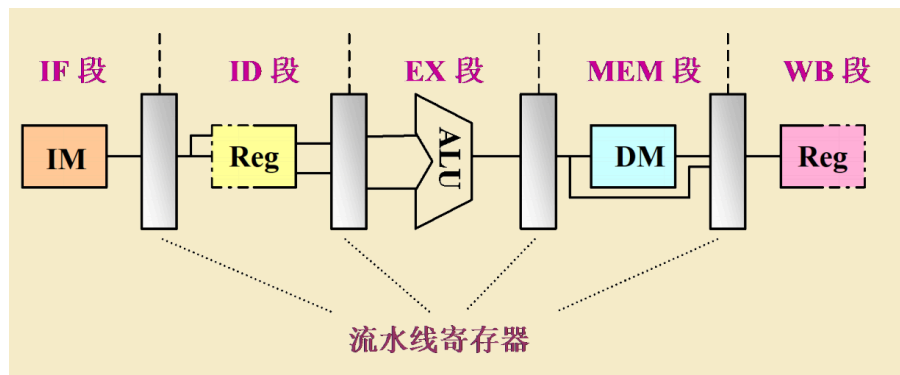
(1) 指令执行的5个周期

1. **取指令周期（IF Instruction Fetch）**
 $IR \leftarrow Mem[PC]$
 $PC = PC + 4$
2. **指令译码/读寄存器周期（ID Instruction decode）**
 - 译码
 - 用IR中的寄存器编号访问通用寄存器组，读出需要的操作数
3. **执行/有效地址计算周期（EX Execution）**
不同指令进行的操作不同，回忆系统硬件的CPU结构原理图
 - **存储器访问指令**: 形成访存的有效地址
 - **寄存器-寄存器ALU指令**: ALU读取寄存器数据进行运算
 - **寄存器-立即数ALU指令**: ALU读取寄存器数据和立即数进行计算
 - **分支指令**: ALU将偏移量与PC相加，形成目标转移地址
4. **存储器访问/分支完成周期（MEM Memory）**
此周期只有**load/store/分支指令**运行，其他指令无任何操作
 - **load指令**: 读取存储器数据
 - **store指令**: 指定数据存放在存储器中
 - **分支指令**: 分支成功，将目标转移地址存放在PC中
5. **写回周期（WB）**
ALU运算指令和load指令 本周期内 将结果写入 **通用寄存器组**

总结

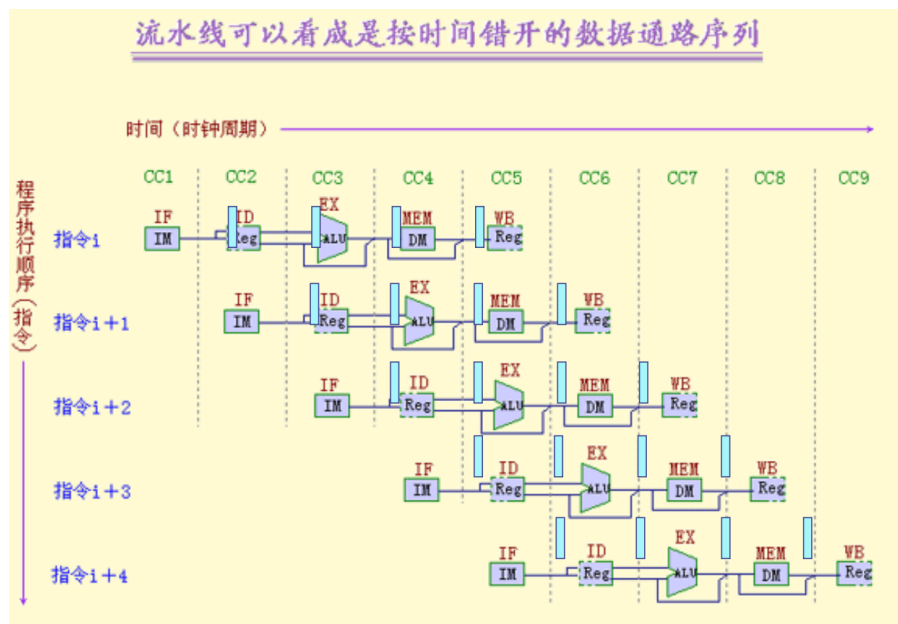
本方案中，分支指令（4个周期，无WB） store指令（4个周期，无WB） 其他周期（5个周期）

(2) 上述方案的流水线实现（知道图的含义即可）



5级流水线的两种描述方法（类时空图法、按时间错开的数据通路序列）

	时钟周期								
指令编号	1	2	3	4	5	6	7	8	9
指令1	IF	ID	EX	MEM	WB				
指令i+1		IF	ID	EX	MEM	WB			
指令i+2			IF	ID	EX	MEM	WB		
指令i+3				IF	ID	EX	MEM	WB	
指令i+4					IF	ID	EX	MEM	WB



3.3.2 流水线相关与冲突

(1) 相关

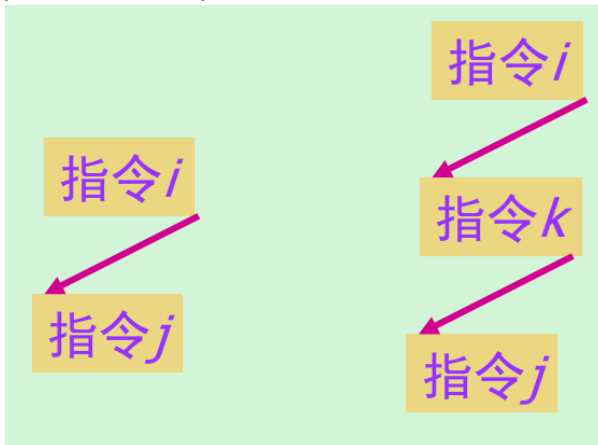
定义：两条指令之间存在某种依赖关系

分类：数据相关、名相关、控制相关

1. 数据相关

指令*i*在前，指令*j*在后 ($i \rightarrow j$)，满足下列条件之一，*j*与*i*数据相关

- *j*使用*i*产生的结果($i \rightarrow j$)
- *j*与*k*数据相关($j \leftarrow k$)，*k*与*i*数据相关($k \leftarrow i$) (传递性)



例如：下面这一段代码存在数据相关。

```
Loop:  L.D    F0, 0(R1)    // F0为数组元素
        ADD.D  F4, F0, F2    // 加上F2中的值
        S.D    F4, 0(R1)    // 保存结果
        DADDIU R1, R1, -8    // 数组指针递减8个字节
        BNE    R1, R2, Loop  // 如果R1≠R2, 则分支
```

Diagram illustrating data dependencies in the code snippet above. Arrows show the flow of data: F0 is used by ADD.D, F4 is used by S.D, and R1 is used by DADDIU and BNE.

2. 名相关

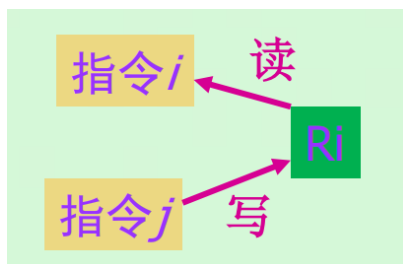
名：指令访问的寄存器/存储单元名称

两条指令*i*、*j*使用相同的名，但没有数据流动，则为名相关

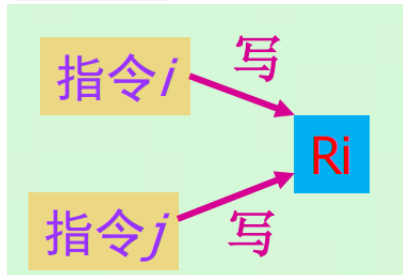
指令*j*与指令*i*之间的名相关有两种 (反相关、输出相关)

关键：和数据相关相比，数据没有在*i*、*j*间流动!!!

反相关： *j*写的名=*i*读的名



输出相关： j、i写相同的名



换名技术： 改变指令中操作数的名来消除名相关，对于寄存器操作数可以为寄存器换名，可以使用编译器静态实现，也可以用硬件动态实现

```
DIV. D    F2, F6, F4
ADD. D    F6, F0, F12
SUB. D    F8, F6, F14
```

DIV. D和ADD. D存在反相关。

进行寄存器换名（F6换成S）后，变成：

```
DIV. D    F2, F6, F4
ADD. D    S, F0, F12
SUB. D    F8, S, F14
```

3. 控制相关

控制相关是由分支指令引起的相关，为了保证程序应有的执行顺序，必须严格按控制相关确定的顺序执行（反映出两条指令之间的分支关联顺序）

控制相关带来了两个限制

1. 与一条分支指令控制相关的指令，不能放到该分支之前（相关不能在前面）
2. 一条指令与某条分支指令无控制相关，不能把这条指令放到分支之后（不相关不能在后面）

控制相关执行顺序必须为：无控制相关指令 → 分支指令 → 控制相关指令

(2) 冲突

定义： 流水线中由于相关的存在，使下一条指令不能在指定时钟周期执行

分类： 结构冲突、数据冲突、控制冲突

带来的问题：

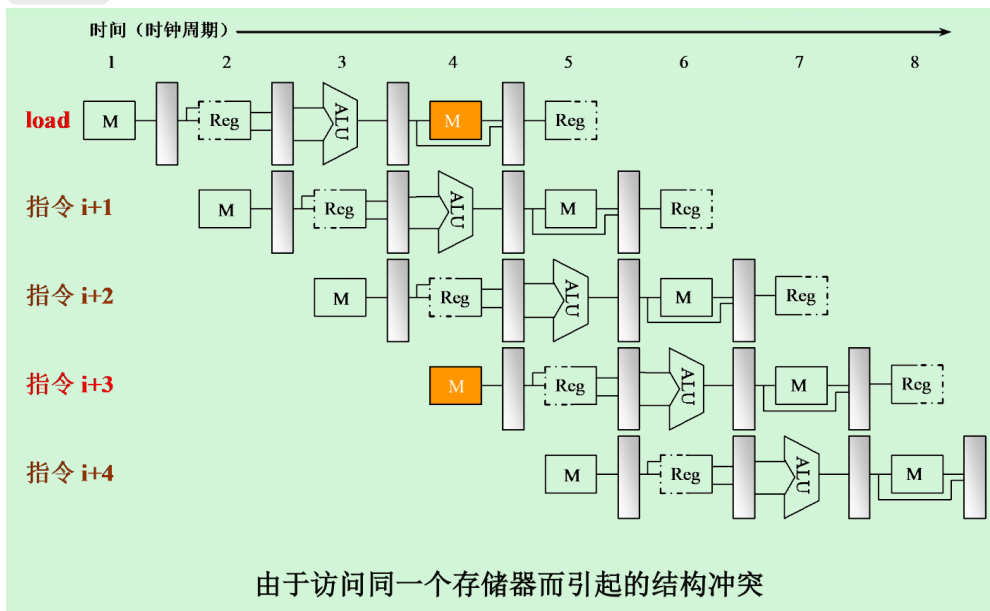
1. 导致错误的运行结果
2. 流水线停顿，降低效率和加速比
3. 约定：一条指令暂停时，该指令后流出的所有指令均暂停，该指令前流出的指令均继续进行（否则无法消除冲突）

1. 结构冲突

产生原因：指令组合因资源冲突无法正常执行（硬件资源满足不了指令重叠执行的要求）；

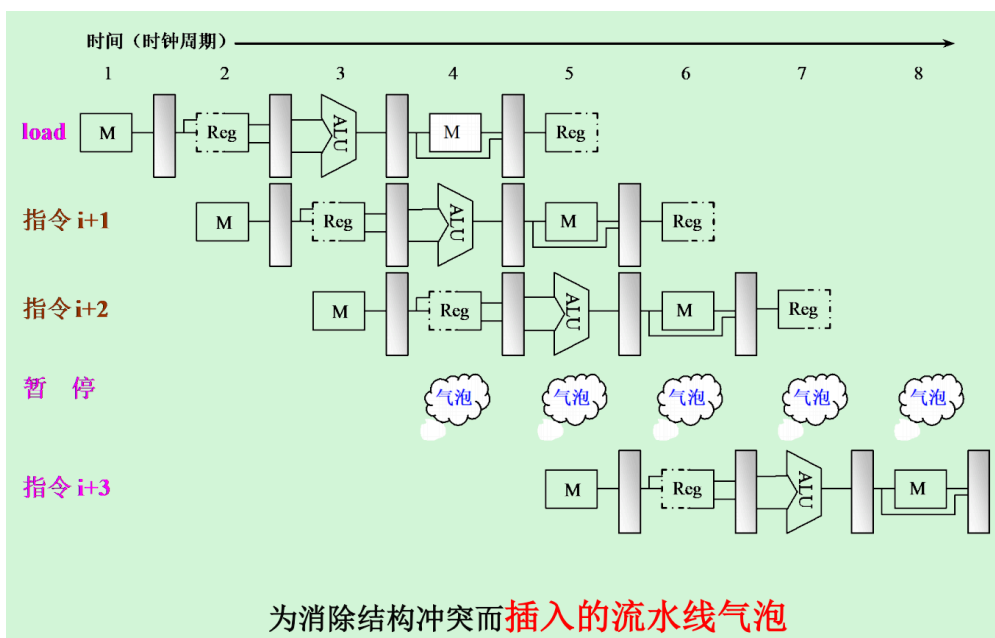
- 常见原因为功能部件不是完全流水、资源份数不够
- （说白了就是调用功能部件冲突了）

例子：访存冲突，只有一个存储器存放数据和指令，故访存指令导致访存冲突



解决方法：

1. 插入暂停周期（流水线气泡）
 - stall后，整体后移一段



引入暂停后的时空图

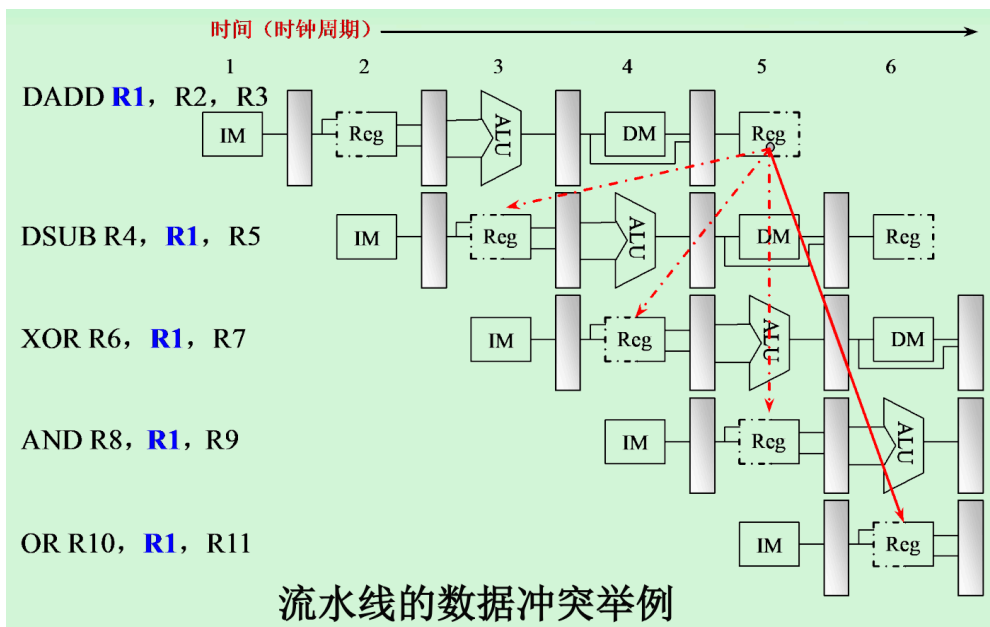
指令编号	时钟周期									
	1	2	3	4	5	6	7	8	9	10
指令i	IF	ID	EX	MEM	WB					
指令i+1		IF	ID	EX	MEM	WB				
指令i+2			IF	ID	EX	MEM	WB			
指令i+3				stall	IF	ID	EX	MEM	WB	
指令i+4						IF	ID	EX	MEM	WB
指令i+5							IF	ID	EX	MEM

2. 设置独立的指令和数据存储器/独立的指令和数据cache

2.数据冲突

产生原因：相关指令在流水线中的重叠执行/重新排序改变了指令读/写操作数的顺序

- （本来好好的，流水线后读写乱套了，就数据冲突了）



类型:

指令i、j, 且i比j早进入流水线i→j

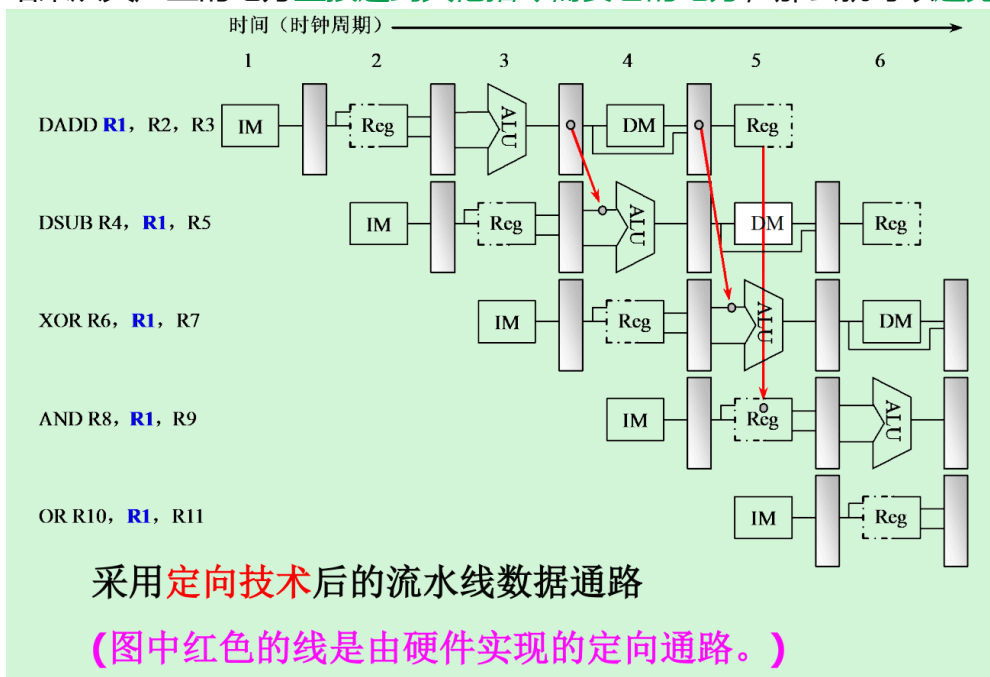
*你应该这么看, i没x 后 j就x 了, 这样就合理了, 后作为中间分割符 (? **

- **写后读冲突 (RAW, read after write)**: i写入前j读取, j读取错误, 一般就发生这种
- **写后写冲突 (RAW, write after write)**: i写入前j写入, 最后结果是i的, 错误
- **读后写冲突 (WAR, write after read)**: i读之前j先写, i读取错误

解决方法:

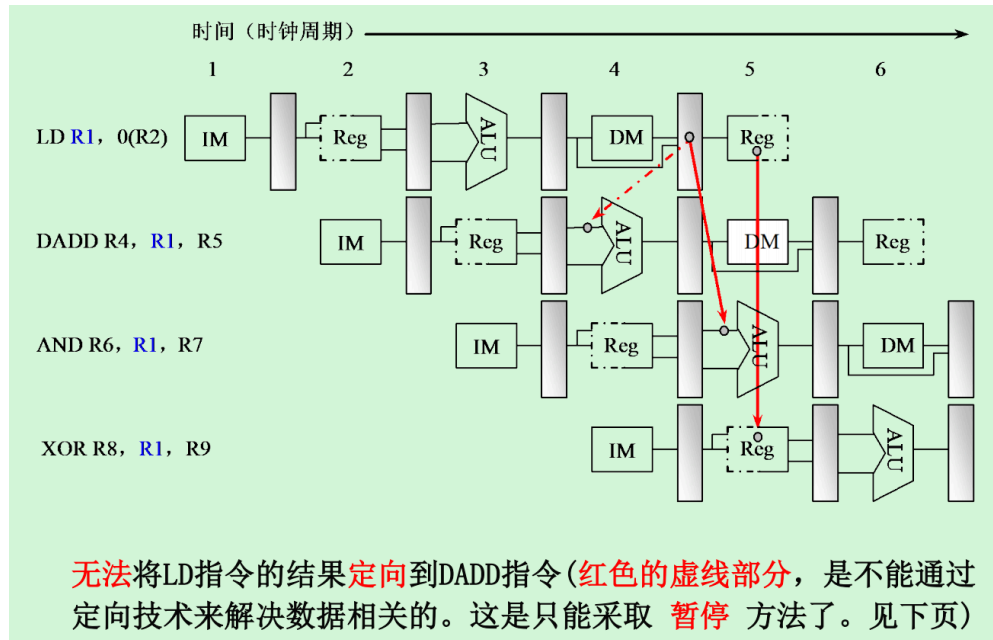
1. 定向技术构建直接数据通路

在某条指令产生计算结果之前, 其他指令并不真正立即需要该计算结果, 如果能够将该计算结果从其产生的地方直接送到其他指令需要它的地方, 那么就可以避免停顿

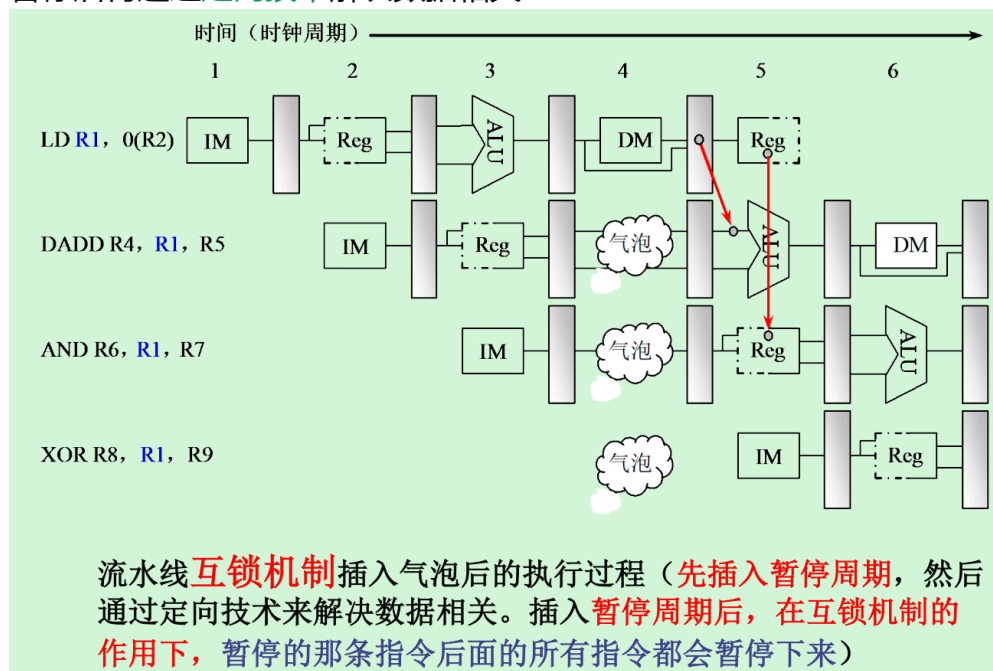


2. 停顿

部分情况无法使用定向技术，只能暂停，**增加流水线互锁硬件**从而插入暂停
例如，图中，DADD来不及提供R1的值就要进行了，所以必须暂停DADD



暂停后再通过**定向技术**解决数据相关



LD R1, 0 (R2)	IF	ID	EX	MEM	WB			
DADD R4, R1, R5		IF	ID	EX	MEM	WB		
AND R6, R1, R7			IF	ID	EX	MEM	WB	
XOR R8, R1, R9				IF	ID	EX	MEM	WB

LD R1, 0 (R2)	IF	ID	EX	MEM	WB			
DADD R4, R1, R5		IF	ID	stall	EX	MEM	WB	
AND R6, R1, R7			IF	stall	ID	EX	MEM	WB
XOR R8, R1, R9				stall	IF	ID	EX	MEM

插入停顿前后的流水线时空图

3. 编译器调度解决数据冲突

让编译器重新组织指令顺序来消除冲突，这种技术称为**指令调度或流水线调度**

举例：

请为下列表达式生成没有暂停的指令序列：

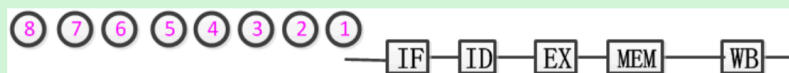
$A = B + C$;

$D = E - F$;

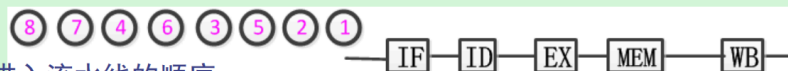
假设载入延迟为1个时钟周期。

题解

调度前的代码	调度后的代码
LD Rb, B	LD Rb, B
LD Rc, C	LD Rc, C
DADD Ra, Rb, Rc	LD Re, E
SD Ra, A	DADD Ra, Rb, Rc
LD Re, E	LD Rf, F
LD Rf, F	SD Ra, A
DSUB Rd, Re, Rf	DSUB Rd, Re, Rf
SD Rd, D	SD Rd, D



调度前送指令进入流水线的顺序。



调度后送指令进入流水线的顺序。

3. 控制冲突

分支的两种结果

- 分支成功：PC改变为目标地址
- 分支失败：PC正常递增

处理分支指令最简单的方法：在分支结果出来前，需要怎么处理？

- 冻结流水线：分支指令后插入暂停周期
- 排空流水线

分支延迟：分支指令引起的延迟

减少分支延迟的方法：

1. 在流水线中尽早判断出分支转移是否成功

2. 尽早计算出分支目标地址

3. 通过软件（编译器）减少分支延迟

1. 预测分支失败：允许分支指令后的指令继续在流水线中流动；

- 如果确定分支失败，将分支指令看作一条普通指令，流水线正常流动；
- 如果确定分支成功，流水线在分支指令之后取出的所有指令转化为空操作，并按分支目的重新取指令执行

流水线对分支的处理过程							
分支指令 i(失败)	IF	ID	EX	MEM	WB		
指令 i+1		IF	ID	EX	MEM	WB	
指令 i+2			IF	ID	EX	MEM	WB
指令 i+3				IF	ID	EX	MEM
指令 i+4					IF	ID	EX

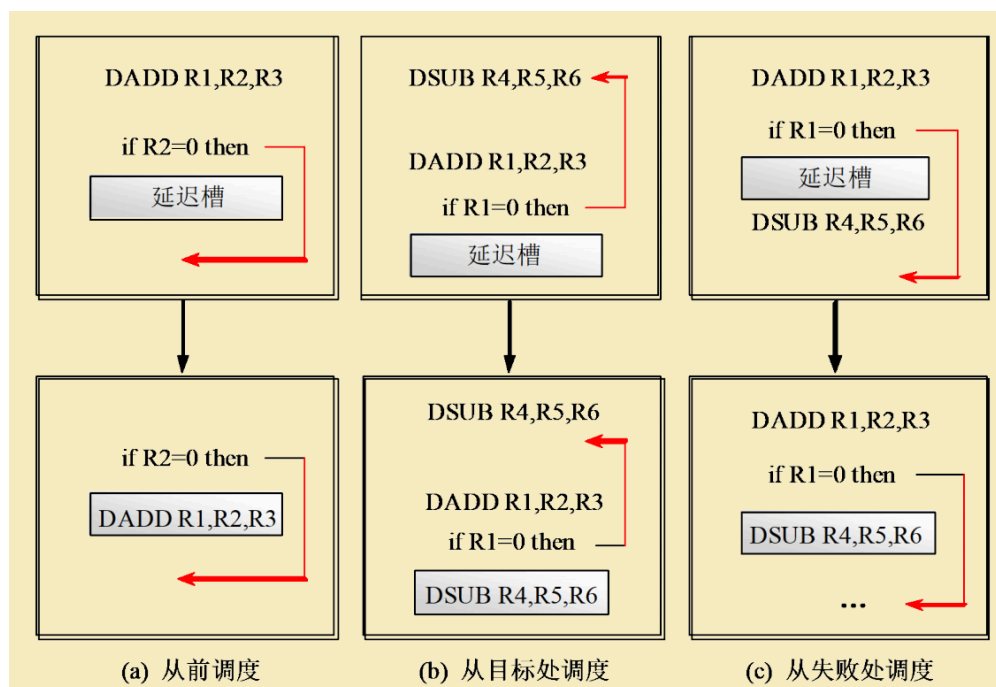
分支指令 i(成功)	IF	ID	EX	MEM	WB		
指令 i+1		IF	ID	idle	idle	idle	
分支目标 j			IF	ID	EX	MEM	WB
分支目标 j+1				IF	ID	EX	MEM
分支目标 j+2					IF	ID	EX

2. 预测分支成功：假设分支转移成功，从分支目标地址处取指令执行

3. 延迟分支：从逻辑上“延长”分支指令的执行时间。延迟分支改为=原来的分支指令+若干个延迟槽，不管分支是否成功，都要按顺序执行延迟槽中的指令

- 编译器把有用的指令调度到延迟槽中，有三种调度方式

- 从前调度
- 从目标处调度
- 从失败处调度



三种方法的要求及效果

调度策略	对调度的要求	什么情况下起作用
从前调度	被调度的指令必须与分支无关	任何情况
从目标处调度	必须保证在分支失败时执行被调度的指令不会导致错误。有可能需要复制指令	分支成功时 (但由于复制指令, 有可能会增大程序空间)
从失败处调度	必须保证在分支成功时执行被调度的指令不会导致错误	分支失败时

改进 (可选) : 分支取消, 预测一致时正常执行, 不一致则延迟槽转化为空

分支失败	分支指令i	IF	ID	EX	MEM	WB				
	延迟槽指令 i+1		IF	idle	idle	idle	idle			
	指令 i+2			IF	ID	EX	MEM	WB		
	指令 i+3				IF	ID	EX	MEM	WB	
	指令 i+4					IF	ID	EX	MEM	WB

分支成功	分支指令i	IF	ID	EX	MEM	WB				
	延迟槽指令 i+1		IF	ID	EX	MEM	WB			
	分支目标指令j			IF	ID	EX	MEM	WB		
	分支目标指令j+1				IF	ID	EX	MEM	WB	
	分支目标指令j+2					IF	ID	EX	MEM	WB

预测分支成功的情况下，分支取消机制的执行情况

3.4 流水线的实现（略）

3.5 向量处理机

3.5.0 标量处理机与向量处理机

向量处理机：流水线处理机中，设置向量数据表示及向量指令

标量处理机：不具有向量数据表示和向量指令的处理机（但是也有流水处理）

3.5.1 向量处理方法

(1) 基本计算方法

向量 $A = a_1, a_2, \dots, a_n$, $B = b_1, b_2, \dots, b_n$

加法： $A + B = (a_1 + b_1, a_2 + b_2, \dots, a_n + b_n)$

乘法： $A \times B = (a_1 b_1, a_2 b_2, \dots, a_n b_n)$

(2) 横向（水平）处理方式

一行一行来，从左到右横向计算

- 先计算: $d_1 \leftarrow a_1 \times (b_1 + c_1)$
- 再计算: $d_2 \leftarrow a_2 \times (b_2 + c_2)$
-
- 最后计算: $d_N \leftarrow a_N \times (b_N + c_N)$

组成循环程序进行处理。

$$k_i \leftarrow b_i + c_i$$

$$d_i \leftarrow k_i \times a_i$$

特点:

1. 数据相关N次, 功能切换2N次
2. 不适合向量处理机的并行处理

(3) 纵向 (垂直) 处理方式

一列一列来, 从上到下纵向进行

向量计算是按列的方式从上到下纵向地进行。

$$\begin{array}{ll} \text{先计算} \left\{ \begin{array}{l} k_1 \leftarrow b_1 + c_1 \\ \dots\dots \\ k_N \leftarrow b_N + c_N \end{array} \right. & \text{再计算} \left\{ \begin{array}{l} d_1 \leftarrow k_1 \times a_1 \\ \dots\dots \\ d_N \leftarrow k_N \times a_N \end{array} \right. \end{array}$$

表示成向量指令:

$$K = B + C$$

$$D = K \times A$$

- 两条向量指令之间:

数据相关: 1次 功能切换: 1次

特点:

1. 数据相关1次, 功能切换1次

(4) 纵横 (分组) 处理方式

向量分为若干组, 组内纵向处理, 各组横向处理

- 先算第1组:

$$k_{1\sim n} \leftarrow b_{1\sim n} + c_{1\sim n}$$

$$d_{1\sim n} \leftarrow k_{1\sim n} \times a_{1\sim n}$$

- 再算第2组:

$$k_{(n+1)\sim 2n} \leftarrow b_{(n+1)\sim 2n} + c_{(n+1)\sim 2n}$$

$$d_{(n+1)\sim 2n} \leftarrow k_{(n+1)\sim 2n} \times a_{(n+1)\sim 2n}$$

- 依次进行下去, 直到最后一组: 第S+1组。
- 每组内各用两条向量指令。

数据相关: 1次 功能切换: 2次

3.5.2 向量处理机的结构(CRAY-1向量处理机)

两种结构: 存储器-存储器结构、寄存器-寄存器结构 (CRAY-1)

CRAY-1结构:

(1) 基本结构 (了解)

功能部件: 12条可并行的单功能流水线; 6个单功能流水部件 (用于向量计算);

向量寄存器V: 512个64位寄存器, 分成8块, 编号V0~V7, 每一块就是一个向量寄存器, 可存放不超过64的向量

标量寄存器S: 8个, S0~S7, 64位

快速暂存器T: 用于标量寄存器和存储器之间的缓冲

向量屏蔽寄存器VM: 64位, 每一位对应向量寄存器的一个单元, 用于向量归并、压缩、还原、测试操作

(2) 向量处理的显著特点 (了解)

1. 各**向量寄存器**均有**连接6个向量功能部件**的单独总线, 各**向量功能部件**也有把运算结果送回**向量寄存器组的总线**
2. 只要不出现 V_i 冲突和功能部件冲突, 各 V_i 均能并行工作, 加快了向量指令的处理。

- **V_i 冲突**: 并行工作的各向量指令的源向量或结果向量使用了相同的 V_i 。

例如: 源向量相同

$$V_3 \leftarrow V_1 + V_2$$

$$V_5 \leftarrow V_4 \wedge V_1$$

- **功能部件冲突**: 并行工作的各向量指令要使用同一个功能部件。

例如: 都需使用乘法功能部件

$$V_3 \leftarrow V_1 \times V_2$$

$$V_5 \leftarrow V_4 \times V_6$$

(3) 指令类型与格式

1. 指令类型

- 向量与向量操作, $V_i \leftarrow V_j \text{ OP } V_k$
- 向量与标量操作, $V_i \leftarrow S_j \text{ OP } V_k$
- 向量取, $V_i \leftarrow \text{存储器}$
- 向量存, $\text{存储器} \leftarrow V_i$

2. ==指令格式 ==

4位	3位	3位	3位	3位
g	h	i	j	k

g, h为操作码, i为目的寄存器编号
j, k为源寄存器编号

4位	3位	3位	3位	3位	16位
g	h	i	j	k	m

g为操作码, h为变址寄存器A的编号
i为目的寄存器编号, j, k, m为形式地址

(4) 向量运算中的相关与冲突

向量运算中的数据相关和功能部件冲突: 采用顺序发射顺序完成方式

1. **写读数据相关**: i指令写的寄存器, j指令读
2. **运算部件冲突**: 两个指令用同一个运算部件
3. **读读数据相关/向量寄存器冲突**: 同时读取同一个向量寄存器



3.5.3 提高向量处理机性能方法

(1) 设置多个功能部件，使其按流水方式并行工作

(2) 链接技术，加快遗传向量指令执行

原理特征：

具有先写后读相关的两条指令，在不出现功能部件冲突和源向量冲突的情况下，可以把功能部件链接起来进行流水处理，以达到加快执行的目的

实质：把流水线定向的思想引入到向量执行过程的结果

例题：第六次课p13,p25拍数计算

总结：

- 结果寄存器可能成为后继指令的操作数
- 寄存器两条有数据相关的向量指令并行执行，这种技术称为两条流水线的链接技术

(3) 采用循环开采技术，加快循环的处理。

(4) 采用多处理机系统，进一步提高性能